

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание 3

Выполнила:

Савченко Анастасия

Группа К3341, Поток БР 1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Реализовать тестирование REST API для сервиса бронирования столиков в ресторанах средствами Postman. Написать тесты внутри Postman. В результате представить комплексный сценарий, охватывающий основной бизнес-процесс приложения.

Ход работы

1. Подготовка тестового окружения

Для тестирования использовался собственный REST API, развёрнутый локально на <http://localhost:3000>. База данных предварительно заполнена тестовыми данными: пять кухонь, три ресторана, столики, блюда и пользователи.

В Postman создана коллекция «Restaurant Booking — Сценарий», содержащая одиннадцать последовательных запросов. Для передачи данных между запросами используются переменные коллекции: token, restaurant_id, table_id, reservation_id, email, password, base_url.

2. Структура сценария

Сценарий моделирует полный путь пользователя: от попытки входа с неверным паролем до отмены бронирования и проверки истории.

Шаг	Название	Метод	Эндпоинт	Ожидаемый код
0	Инициализация	GET	/cuisines	200
1	Вход — неверный пароль	POST	/auth/login	401

Шаг	Название	Метод	Эндпоинт	Ожидае мый код
2	Вход — успешно	POST	/auth/login	200
3	Профиль пользователя	GET	/users/me	200
4	История бронирований	GET	/reservations	200
5	Поиск ресторанов с фильтрами	GET	/restaurants	200
6	Меню выбранного ресторана	GET	/restaurants/{id}/dishes	200
7	Доступные столики	GET	/restaurants/{id}/tables/available	200
8	Создание бронирования	POST	/reservations	201
9	Отмена бронирования	PATC H	/reservations/{id}/cancel	200
10	История после отмены	GET	/reservations	200

3. Описание шагов сценария

Шаг 0. Инициализация. Выполняется GET-запрос к /cuisines. В блоке Tests очищаются все переменные коллекции — это гарантирует что сценарий стартует с чистого состояния независимо от предыдущих запусков.

Шаг 1. Неудачный вход. Отправляется POST-запрос к /auth/login с намеренно неверным паролем. Сервер возвращает 401 и код ошибки UNAUTHORIZED. Тест проверяет что access_token в ответе отсутствует.

Шаг 2. Успешный вход. Отправляется POST-запрос к /auth/login с корректными учётными данными. Сервер возвращает 200, access_token и refresh_token. Токен сохраняется в переменную коллекции token для использования в последующих защищённых запросах.

Шаг 3. Профиль пользователя. GET-запрос к /users/me с заголовком Authorization. Проверяется что email в ответе совпадает с email из переменной коллекции.

Шаг 4. История бронирований. GET-запрос к /reservations. Проверяется структура ответа: наличие массива data.

Шаг 5. Поиск ресторанов с фильтрами. GET-запрос к /restaurants с параметрами: cuisine_name=Итальянская, city_name=Санкт-Петербург, price_tier=mid. Каждый ресторан в ответе проверяется на соответствие всем трём фильтрам. ID первого найденного ресторана сохраняется в переменную restaurant_id.

Шаг 6. Меню выбранного ресторана. GET-запрос к /restaurants/{{restaurant_id}}/dishes. Проверяется что меню содержит массив блюд.

Шаг 7. Доступные столики. GET-запрос к /restaurants/{{restaurant_id}}/tables/available с параметрами даты, времени

и количества гостей. Проверяется что есть хотя бы один доступный столик. ID первого столика сохраняется в переменную `table_id`.

Шаг 8. Создание бронирования. POST-запрос к `/reservations` с телом, содержащим сохранённые `restaurant_id` и `table_id`, дату, время и количество гостей. Сервер возвращает 201 и статус `pending`. ID бронирования сохраняется в переменную `reservation_id`.

Шаг 9. Отмена бронирования. PATCH-запрос к `/reservations/{reservation_id}/cancel`. Проверяется что статус изменился на `cancelled`.

Шаг 10. История после отмены. Повторный GET-запрос к `/reservations`. Проверяется что в истории есть хотя бы одна отменённая бронь.

4. Результаты тестирования

Все одиннадцать шагов сценария выполняются успешно при последовательном запуске коллекции. Переменные корректно передаются между запросами, что обеспечивает целостность сценария. Сценарий покрывает основной бизнес-процесс: аутентификация → поиск → бронирование → отмена.

Коды ответов соответствуют спецификации OpenAPI из Д32. Обработка ошибок работает корректно: неверный пароль возвращает 401, конфликт бронирования — 409, несуществующий ресурс — 404.

5. Дополнительные тесты

Помимо основного сценария, в файле **`backend_app.postman_collection.json`** содержится расширенный набор тестов (23 запроса), покрывающий все эндпоинты API с различными кодами ответов:

- Успешные операции (200, 201, 204)
- Ошибки валидации (400, 422)
- Ошибки аутентификации (401)

- Ошибки доступа (403)
- Ресурс не найден (404)
- Конфликты (409)
- Фильтрация с различными комбинациями параметров
- Пагинация

Эти тесты предназначены для обособленного запуска и не используют общие переменные с очисткой, поэтому при едином прогоне коллекции могут возникать конфликты переменных. Каждый кейс в этом файле самодостаточен и проверяет конкретный аспект API.

Вывод

В результате выполнения домашнего задания разработан комплексный сценарий тестирования API сервиса бронирования столиков в ресторанах. Сценарий реализован в виде коллекции Postman из одиннадцати последовательных шагов и охватывает полный бизнес-процесс: от входа в систему до отмены бронирования. Все тесты содержат встроенные проверки (assertions) на JavaScript, валидирующие коды ответов, структуру JSON и бизнес-логику.

Дополнительно подготовлен расширенный набор из тестов, покрывающих все эндпоинты API и возможные коды ошибок. Оба файла коллекций приложены к отчёту в репозитории.